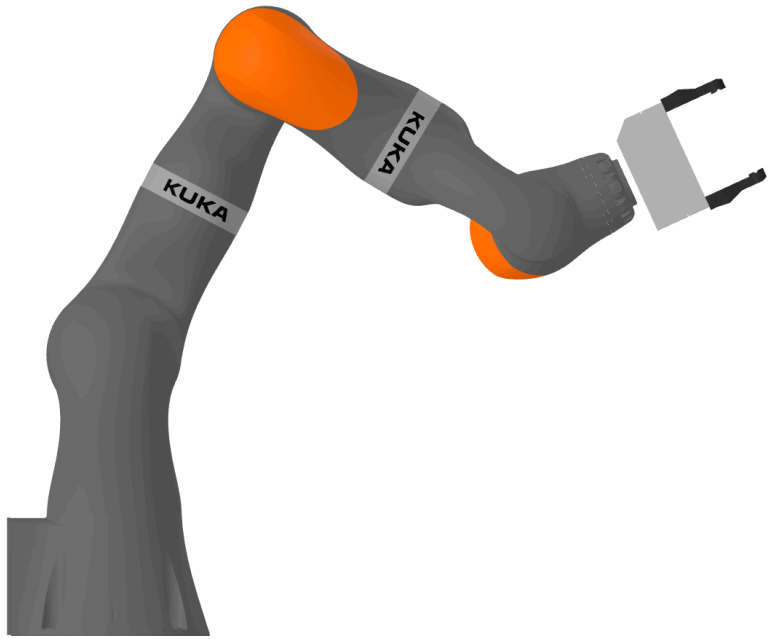


Model-based Learning

MIT 6.4210/2

Robotic Manipulation

Fall 2026, Lecture 12

Massachusetts Institute of Technology

Reinforcement learning

Lecture 12: model-based RL

Leslie Kaelbling

6.4210 Fall 2026

Learning a model

Policies answer “what should I do?”
Models answer “what would happen if I did this?”

- Predicted next states let us compare possible futures before acting
- Changing the reward requires re-learning a policy or value function
- A transition model can be reused when the task changes but the dynamics do not

$T(s, a, s') = P(s' \mid s, a)$: transition model $R(s, a)$: immediate reward

Goal-conditioned reinforcement learning

The goal becomes another input: $\pi(a \mid s, g)$ and $Q(s, a, g)$

$$R(s, a, g) = \begin{cases} 0, & s \in \mathcal{G}(g) \\ -C(s, a), & s \notin \mathcal{G}(g) \end{cases}$$

- In a stochastic shortest-path problem, $\gamma = 1$ and reaching the goal ends the episode
- We pay for the action that enters the goal set, then receive no further rewards
- Maximizing return minimizes the expected cost of reaching the goal

g: goal description $\quad \mathcal{G}(g)$: states satisfying it $\quad C(s, a)$: step cost
Assume nonterminal costs $\geqslant c_{\min} > 0$ and a policy with finite expected total cost

4

Reusing experience

If the physical dynamics stay the same, an off-policy learner can relabel

$$((s, g_1), a, r, (s', g_1)) \quad \longrightarrow \quad ((s, g_2), a, R(s, a, g_2), (s', g_2))$$

- Recompute both the reward and whether s' is terminal for g_2
- Omit transitions starting at an already-satisfied goal
- *Hindsight experience replay* relabels with goals actually reached

Goal-reaching costs as quasimetrics

For exact state goals, minimum expected total cost gives a distance:

$$d(s, g) = -V^*(s, g), \qquad d(s, g) \leqslant d(s, u) + d(u, g)$$

- Nonnegative, zero exactly when $s = g$, and obeys the triangle inequality
- Need not be symmetric: dropping an object is easier than retrieving it
- Quasimetric learning builds this structure into a learned representation

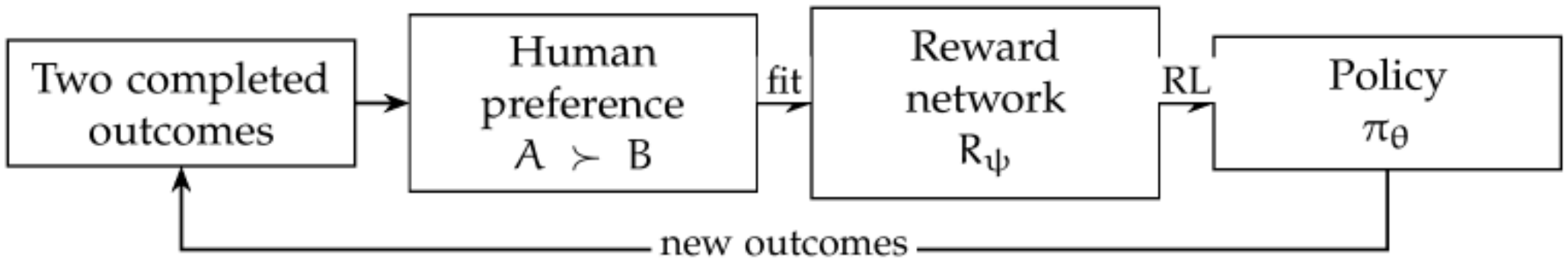
These relationships can help us choose intermediate goals

Same dynamics and positive step costs for all goals; undiscounted return
Finite distances on mutually reachable states; otherwise d may be infinite
Wang et al., Optimal Goal-Reaching RL via Quasimetric Learning

Learning a reward model

"Set the table nicely" may be easier to judge than to write as a reward

- People can label success, give scores, or compare completed outcomes
- Supervised learning fits a reward network; RL then uses its predictions



R_ψ: reward network π_θ: policy
Christiano et al., Deep Reinforcement Learning from Human Preferences

Comparing two final states

For two outcomes of the same task, the network predicts scores

$$r_A = R_\psi(s_A), \qquad r_B = R_\psi(s_B)$$

The *Bradley–Terry model* predicts the person's preference:

$$p = p_\psi(A \succ B) = \frac{e^{r_A}}{e^{r_A} + e^{r_B}} = \frac{1}{1 + e^{-(r_A - r_B)}}$$

- Only the score difference matters; equal scores give probability 1/2
- The training example tells us which outcome the person prefers

s_A, s_B: final states $A \succ B$: person prefers A
These scores are terminal rewards, rather than sums along trajectories

A reward update from one preference

Suppose the person prefers A, and write $\Delta = r_A - r_B$

$$\mathcal{L}(\psi) = -\log p = \log(1 + e^{-\Delta}), \qquad \frac{\partial \mathcal{L}}{\partial \Delta} = p - 1$$

$$\nabla_\psi \mathcal{L} = (p-1)\left[\nabla_\psi R_\psi(s_A) - \nabla_\psi R_\psi(s_B)\right]$$

$$\psi \leftarrow \psi + \alpha(1-p)\left[\nabla_\psi R_\psi(s_A) - \nabla_\psi R_\psi(s_B)\right]$$

Backpropagation through the same network supplies both score gradients
The update is larger when the preference was poorly predicted

$\alpha > 0$: learning rate All right-hand quantities are evaluated before the update

Inverse reinforcement learning

Behavior cloning fits actions; *IRL* fits a cost that could explain them

$$p_\lambda(\tau) = \frac{\exp[-C_\lambda(\tau)]}{Z(\lambda)}, \qquad Z(\lambda) = \sum_{\tau' \in \mathcal{F}} \exp[-C_\lambda(\tau')]$$

- Cheaper feasible trajectories are more likely, but not certain
- We fit the cost by maximizing demonstration likelihood
- The learned cost can guide planning in new situations; demonstrations generally do not identify it uniquely

$\mathcal{F}$: feasible trajectories λ: cost parameters Z: normalizer
With two alternatives and reward equal to negative cost, this is Bradley–Terry

Sequential IRL: predicting behavior

Features such as energy use or obstacle proximity give a linear step cost:

$$c_\lambda(s, a) = \lambda^\top \phi(s, a)$$

Known deterministic dynamics, fixed start and horizon H, finite actions
A backward pass computes soft *cost-to-go* values, with $V_H(s) = 0$:

$$Q_t(s, a) = c_\lambda(s, a) + V_{t+1}(f(s, a))$$

$$V_t(s) = -\log \sum_a e^{-Q_t(s,a)}, \qquad \pi_t(a \mid s) = e^{V_t(s) - Q_t(s,a)}$$

This policy generates the Boltzmann distribution over complete trajectories

$\phi(s, a)$: known features λ: learned weights f: known transition function
Here Q_t, V_t are soft costs, not reward values; no terminal cost, temperature 1

Sequential IRL: updating the cost

Total trajectory features determine total cost:

$$\Phi(\tau) = \sum_{t=0}^{H-1} \phi(s_t, a_t), \qquad C_\lambda(\tau) = \lambda^T \Phi(\tau)$$

A forward pass under π_t predicts expected feature counts

$$\lambda \leftarrow \lambda - \alpha\big(\bar{\Phi}_D - \mathbb{E}_{p_\lambda}[\Phi(\tau)]\big)$$

- A feature that occurs too often in predicted behavior gets a larger cost
- State indicators compare expected and demonstrated visit counts

$\bar{\Phi}_D$: average total features in demonstrations α: learning rate
Ziebart et al., Maximum Entropy IRL; for stochastic dynamics: maximum causal entropy IRL

Learning transition models

Recorded transitions $\mathcal{D} = \{(s_i, a_i, s'_i)\}_{i=1}^{N}$ supply supervised examples

$$\mathcal{L}_{\text{reg}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|s'_i - f_\theta(s_i, a_i)\|^2$$

Squared loss fits a conditional mean, which may not be a likely outcome

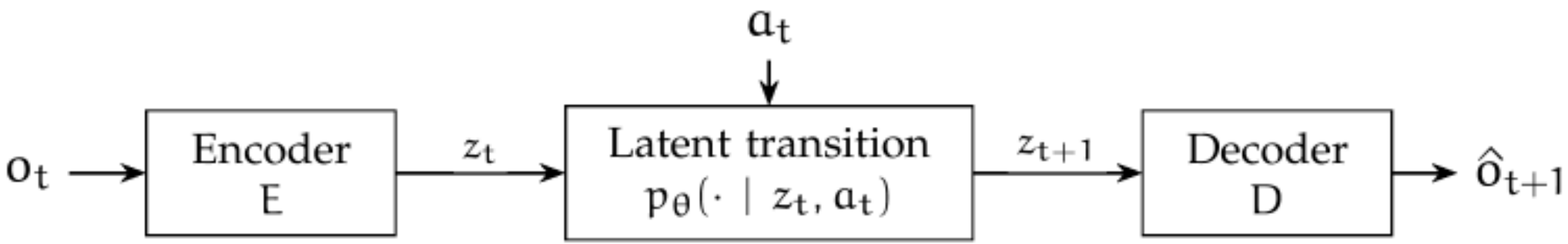
$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(s'_i \mid s_i, a_i)$$

Categorical or Gaussian models describe stochastic outcomes
Conditional diffusion and flow models can represent richer distributions

f_θ: deterministic prediction p_θ: learned transition distribution
Expected negative log-likelihood is minimized by the true distribution, if representable

Latent world models

An encoder compresses observations; dynamics predict in the latent space



- Reconstruction and prediction losses train representation and dynamics
- Latent states must support prediction of observations and rewards
- Partial observations may require an encoder with memory

Reconstructing an image well does not guarantee a useful state for control

o_t: observation z_t: latent state E: encoder D: decoder

Video plans and inverse dynamics

A video model proposes a visual plan

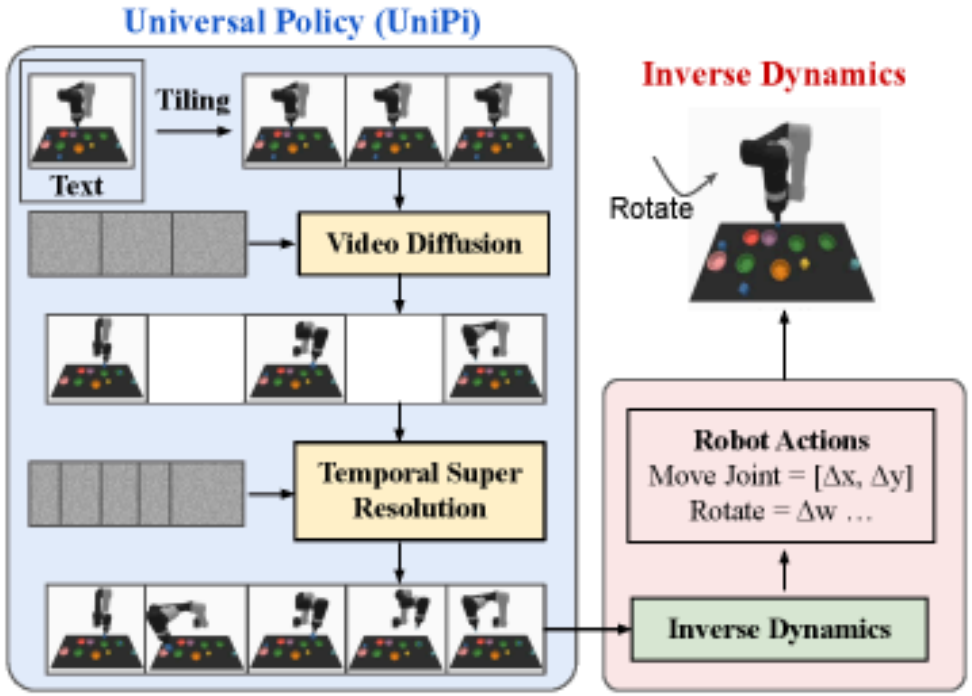
$$o_{1:H} \sim p_\theta(\cdot \mid o_0, g)$$

Inverse dynamics predicts actions

$$a_t \sim q_\psi(\cdot \mid o_t, o_{t+1})$$

It needs recorded (o_t, a_t, o_{t+1}) triples

A plausible-looking video need not be an executable plan



g: goal description o_0: current image
Du et al., UniPi, Fig. 2, cropped from the paper, CC BY 4.0

Learning a policy or planning now

Policy learning moves computation into training

- A learned policy makes action selection cheap
- Its training may not cover every state and goal we will encounter

Inference-time planning concentrates computation on the current situation

- Use the model to choose an action, execute it, observe, and plan again
- Model predictive control (MPC) plans over a finite horizon each time

Planning only the next action still need not be computationally easy

Approximations in planning

- **Finite horizon:** truncate the future, optionally adding a terminal value
- **Sampled outcomes:** use a few next-state samples to represent uncertainty
- **Determinization:** use one predicted outcome, discarding contingencies
- **Sampled actions:** search a finite set of continuous-action candidates
- **Partial observability:** approximate beliefs and future observations

Replanning lets us revise later choices when new information arrives

A finite-horizon MPC objective

Search over an action sequence $\mathbf{a} = (a_0, \ldots, a_{H-1})$:

$$\mathbf{a}^* = \arg\max_{\mathbf{a} \in \mathcal{A}^H} \mathbb{E}\left[\sum_{k=0}^{H-1} \gamma^k R(s_k, a_k) + \gamma^H \hat{V}(s_H)\right]$$

- Rollouts start at the current state estimate s_0
- Sample $s_{k+1} \sim p_\theta(\cdot \mid s_k, a_k)$, or use $f_\theta(s_k, a_k)$
- Execute only a_0^*, observe the outcome, and solve again

We seek the best sequence that cannot depend on outcomes (*open loop*)

H: horizon $\hat{V}$: return estimate beyond H $\mathcal{A}$: allowed actions
The goal is fixed and suppressed in R and $\hat{V}$; optimality here is within the model

Shooting and the cross-entropy method

Random shooting samples M action sequences and selects the best score

$$\hat{J}(\mathbf{a}) = \frac{1}{K}\sum_{j=1}^{K}\left[\sum_{k=0}^{H-1}\gamma^k R(s_k^{(j)}, a_k) + \gamma^H \hat{V}(s_H^{(j)})\right]$$

The *cross-entropy method* (CEM) improves the sampling distribution:

1. Sample candidate sequences and score their model rollouts
2. Keep an elite fraction with the highest scores
3. Refit the proposal to the elites and repeat

For continuous actions, often refit a Gaussian mean and covariance

$J(\mathbf{a})$: expected MPC return $\hat{J}$: estimate using K rollouts per candidate
Chua et al., Deep RL in a Handful of Trials using Probabilistic Dynamics Models

Gradient-based trajectory optimization

For continuous actions, differentiate through a deterministic rollout:

$$s_{k+1} = f_\theta(s_k, a_k), \qquad \mathbf{a} \leftarrow \mathbf{a} + \alpha \nabla_{\mathbf{a}} J(\mathbf{a})$$

- f_θ must be differentiable in its state and action inputs
- Reward and terminal value must also be differentiable along the rollout
- The actions change during optimization; the model parameters stay fixed

Optimization can **adversarially exploit model errors**: high predicted return, poor real behavior

$\alpha > 0$: action-optimization step size Nonconvex objective: initialization matters

Model predictive path integral control

MPPI samples $\mathbf{a}^{(i)} = \mathbf{u} + \boldsymbol{\epsilon}^{(i)}$ from a Gaussian proposal $q_{\mathbf{u}}$ with fixed covariance

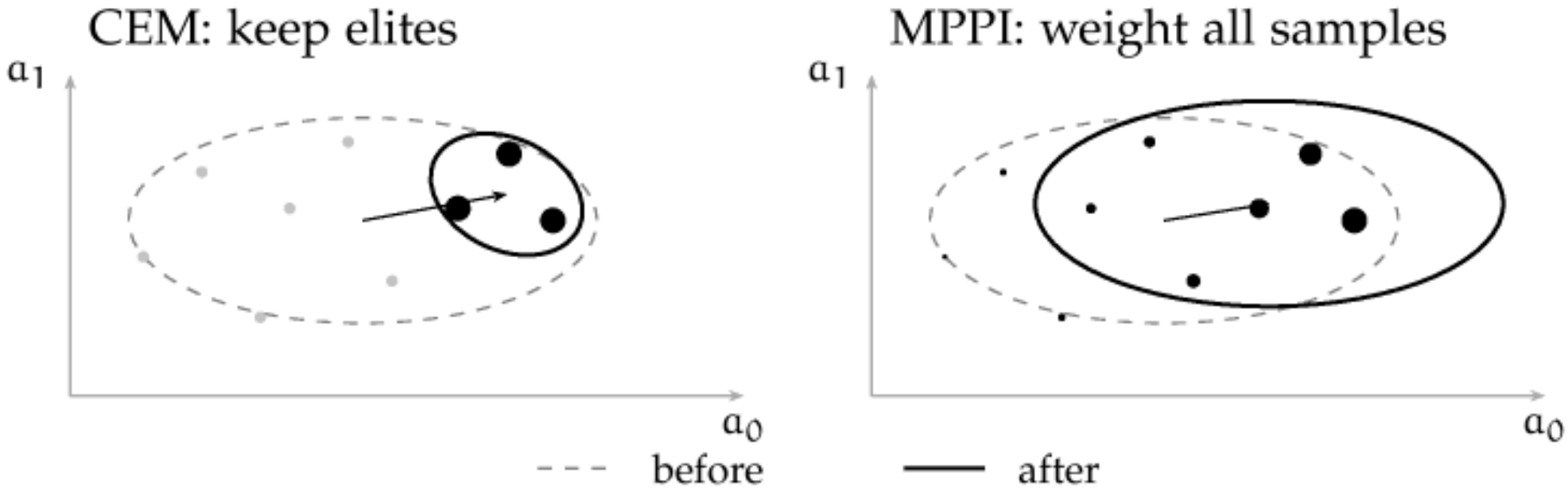
$$w_i \propto \exp\left(\hat{J}(\mathbf{a}^{(i)})/\eta\right) \frac{q_0(\mathbf{a}^{(i)})}{q_{\mathbf{u}}(\mathbf{a}^{(i)})}, \qquad \sum_i w_i = 1$$

$$u_k \leftarrow u_k + \sum_i w_i \epsilon_k^{(i)}$$

- Lower temperature puts more weight on high-return candidates
- The density ratio corrects for sampling around the current plan

$\mathbf{u}$: nominal sequence $\eta > 0$: temperature q_0: reference distribution
Williams et al., Information Theoretic Model Predictive Control

CEM and MPPI: updating a proposal



Each dot is a two-action sequence (a_0, a_1)

- CEM refits mean and covariance to equally weighted elites
- MPPI uses all samples; dot area shows weight and covariance stays fixed

Monte Carlo tree search

A tree represents action choices and their possible outcomes

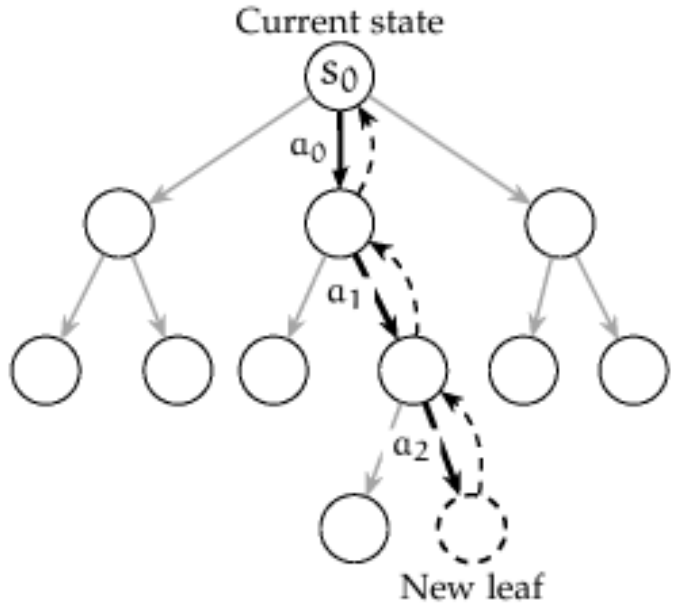
$$a = \arg\max_{a} \left[\bar{Q}(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right]$$

- High average return makes an action attractive
- Limited sampling adds an exploration bonus; unvisited actions go first
- This selection rule is called *UCT*

$\bar{Q}(s, a)$: mean simulated return $N(s, a)$: edge visits $N(s) = \sum_a N(s, a)$
$c > 0$: exploration constant Kocsis and Szepesvári, Bandit Based Monte-Carlo Planning

MCTS simulations and contingencies

1. Follow selected actions through the transition model
2. Expand a new node and estimate its return with a rollout or $\hat{V}$
3. Back up along the path: $G \leftarrow R(s, a) + \gamma G$
4. Update visit counts and average returns; repeat until the budget runs out
5. Execute a root action, commonly the most visited one



A tree allows different continuations after different sampled outcomes

Nodes: states Solid path: selected actions Dashed arrows: return backup
Deterministic example shown; stochastic dynamics can branch on outcomes as well as actions

Continuous actions and progressive widening

A heavily visited node can support more candidate actions

$$m(s) = \max\{1, \lceil k_a N(s)^{\beta} \rceil\}, \qquad k_a > 0, \quad 0 < \beta < 1$$

- Below this limit, propose a new action; otherwise select an existing one
- Sublinear growth leaves time to evaluate each candidate repeatedly
- *Double progressive widening* also limits sampled outcome children under continuous stochastic dynamics

$m(s)$: allowed action children k_a, β: widening parameters
Couëtoux et al., Continuous Upper Confidence Trees

AlphaZero: learning and planning improve each other

Learning improves planning

- A learned policy supplies action priors that focus MCTS
- A learned value function evaluates leaves without finishing every game

Planning improves learning

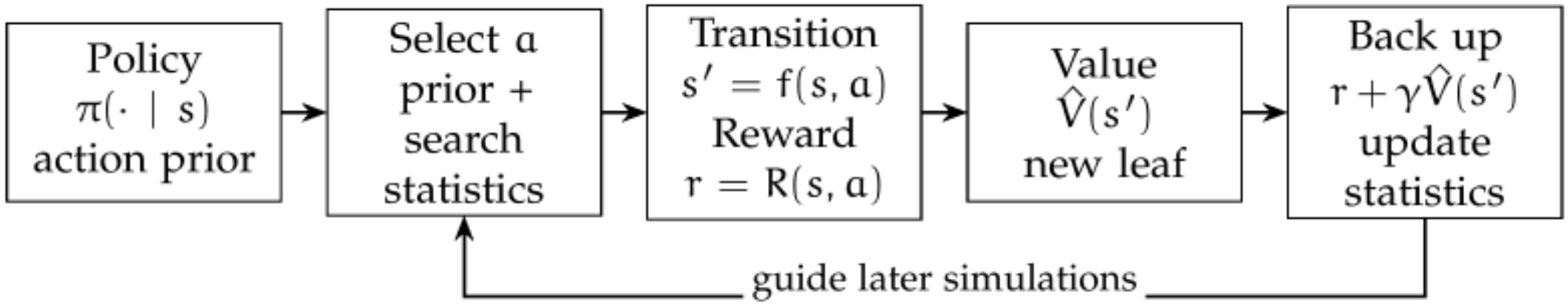
- Search visit counts provide improved policy-training targets
- Outcomes of search-driven self-play games train the value function

AlphaZero uses an exact transition model: it is given the game rules

Silver et al., Mastering Chess and Shogi by Self-Play with a General RL Algorithm

MuZero: learning the dynamics too

An encoder maps observation history to a latent search state

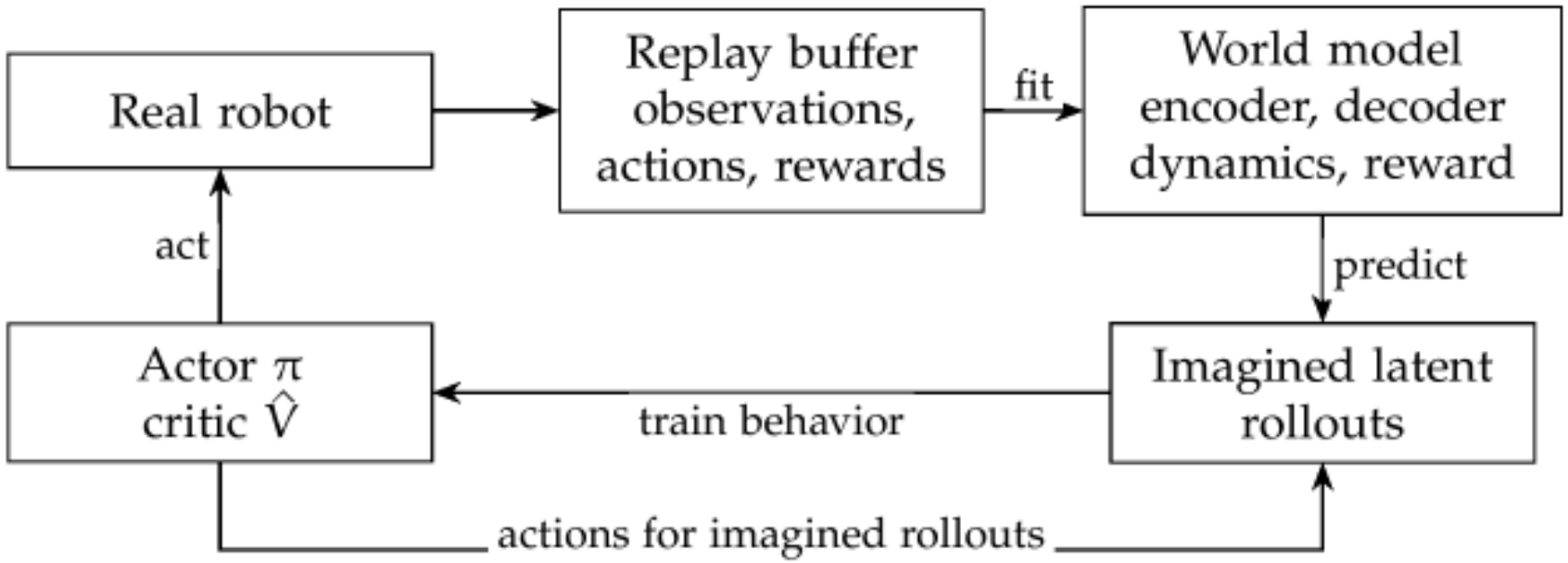


Observed rewards, return targets, and search policies train the networks
Latent dynamics support planning without reconstructing observations

One new leaf and its one-edge backup are shown; returns propagate to ancestors
Schrittwieser et al., Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model

DayDreamer: learning through imagined experience

Dreamer trains an actor and critic inside a learned world model
DayDreamer applies this loop to physical robots



Real experience trains the model; imagined latent rollouts train behavior
Wu et al., DayDreamer: World Models for Physical Robot Learning

Dreamer: model, behavior, and execution

- **World model:** an encoder estimates latent state from observations and history; dynamics and reward heads predict its evolution
- **Model training:** reconstruction, reward prediction, and agreement between inferred and predicted latent states
- **Behavior training:** imagined rewards and bootstrapped values train the critic; the actor maximizes predicted returns
- **Execution:** update the latent state and call the actor, without online tree search

For continuous actions, actor gradients pass through the dynamics
World-model parameters stay fixed during behavior updates